

# Machine Learning

Winter 2011/12

Roland Memisevic

Lecture 4, Nov. 14, 2011

## Non-linear basis functions

- ▶ In the preceding lectures we considered linear models

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} = \sum_j w_j x_j$$

for use in regression or in classification tasks. (For the latter we used these as discriminant functions which we plugged into “squashing” functions).

- ▶ We saw that we can turn these into non-linear models by projecting inputs onto fixed, non-linear *basis functions*:

$$y(\mathbf{x}) = \mathbf{w}^T \Phi(\mathbf{x}) = \sum_j w_j \phi(x_j)$$

- ▶ In this lecture, we will consider *learning* the basis functions along with the model parameters from data.

## Outline

- ▶ The limits of fixed basis functions.
- ▶ Neural networks.
- ▶ Error backpropagation (“backprop”).

## The limits of fixed basis functions

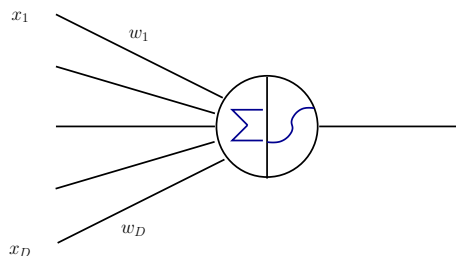
- ▶ Learning the basis functions can be advantageous over using fixed ones:
- ▶ Often it is not clear which basis functions are the most suitable for a given task.
- ▶ Even then, we could in principle use a lot of basis functions and let the model figure out which ones are good, but this will be prone to overfitting and unstable.

## Feed-forward neural networks

- ▶ The most common type of non-linear model is the **feed-forward neural network**, **multi-layer perceptron** or **backprop network**.
- ▶ Here, the components (dimensions) of the basis functions are modeled as artificial “neurons” that are combined into non-linear networks.
- ▶ The response of each single node (“neuron”) in the network is a linear function that is subjected to a (scalar) non-linear transformation  $h$ :

$$h(\mathbf{w}^T \mathbf{x})$$

## Feed-forward neural networks



- ▶ Motivation: The idealized neuron computes a weighted sum of its inputs and fires when the sum reaches a threshold.
- ▶ Its average firing rate can be described with a soft-threshold, such as a sigmoid.

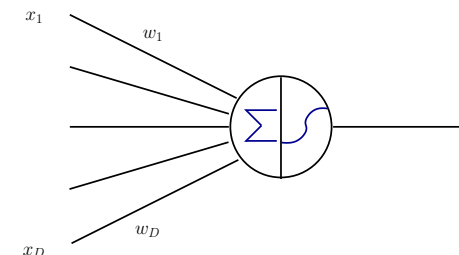
## Feed-forward neural networks

- ▶ This can be thought of as an abstract model of a biological neuron, which collects incoming activity and has a “firing rate” that depends non-linearly on this activity.
- ▶ The non-linear function  $h()$  is often called “**activation function**”. The most commonly used activation function is the sigmoid:

$$h(a) = \sigma(a) = \frac{1}{1 + \exp(-a)}$$

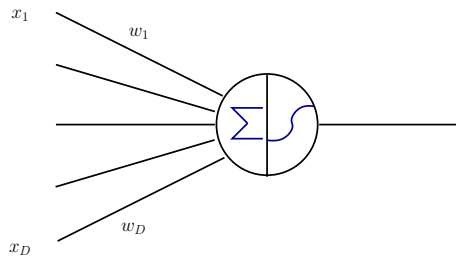
- ▶ The sigmoid behaves like the average of a “noisy threshold”.

## Aside: The perceptron



- ▶ Note that, if we replace the output soft-threshold with a hard threshold (eg., sign-function), we obtain a linear classifier!
- ▶ This model is known as **Perceptron** (Rosenblatt, 1957).
- ▶ It is one of the earliest biologically-inspired classifiers, and it comes with a simple learning-rule:

## Aside: The perceptron

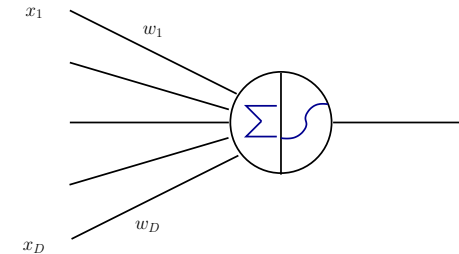


### Perceptron Learning Rule

Encode class-labels as either  $-1$  or  $1$ . Iterate over training cases  $\{(\mathbf{x}_n, t_n)\}$  (in any order), and

- ▶ set  $\mathbf{w} \leftarrow \mathbf{w} + (t_n - \text{sgn}(\mathbf{w}^T \mathbf{x}_n)) \mathbf{x}_n$

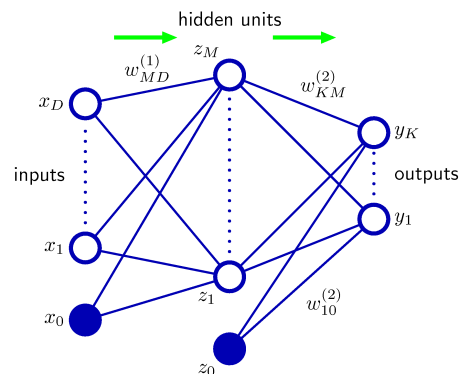
## Aside: The perceptron



- ▶ Several variants of the perceptron learning rule exist.
- ▶ Note that for correctly classified inputs, it does nothing.
- ▶ The learning rule provably converges, if the data-set is linearly separable.
- ▶ But it is only a linear model. So we return to soft-thresholds and networks of neurons:

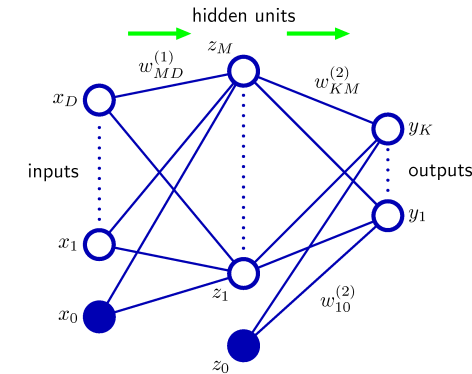
— End Aside —

## Feed-forward neural networks



- ▶ We can now combine many of these computational units into large non-linear networks.
- ▶ Here,  $z_i$  denote the outputs of the first layer of units,  $y_i$  the outputs of the last layer, and  $w_{ji}^{(k)}$  are the parameters for the  $i$ 'th unit in layer  $k$ .

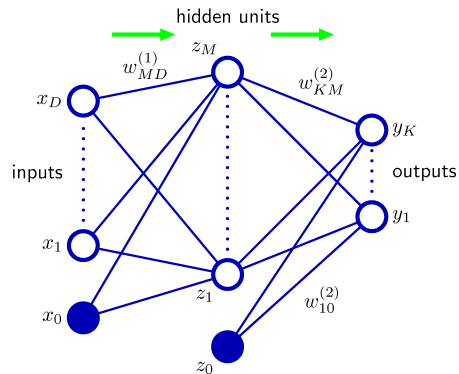
## Feed-forward neural networks



- ▶ The computation that this network performs can be written formally

$$y_k(\mathbf{x}) = \sum_{j=1}^M w_{kj}^{(2)} h \left( \sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)} \right) + w_{k0}^{(2)}$$

## Feed-forward neural networks



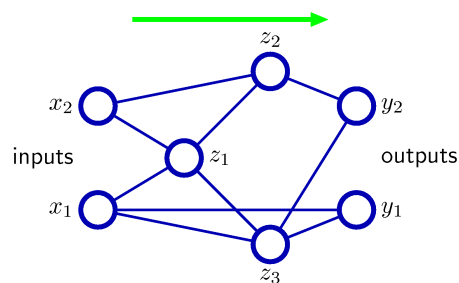
- ▶ We can absorb bias terms (including intermediate biases) as usual to simplify:

$$y_k(\mathbf{x}) = \sum_{j=0}^M w_{kj}^{(2)} h\left(\sum_{i=0}^D w_{ji}^{(1)} x_i\right)$$

## Feed-forward neural networks

- ▶ All layers except for the first (inputs) and last (outputs) are often referred to as “**hidden layer**” and their components as “**hidden units**”.
- ▶ The parameters are also referred to as “**weights**”.
- ▶ In principle, we could define more complex architectures than layered networks. But learning would then get much more complicated.
- ▶ Note, in particular, that given an input vector  $\mathbf{x}$ , the network output  $\mathbf{y}$  is now well-defined. It wouldn't, if we would allow for connections feeding back into previous layers, for example.

## Feed-forward neural networks



- ▶ A network may have sparse connectivity, where most connections are zero, leading to more interesting topologies like the one shown.
- ▶ In general, all we have to make sure is that model is a “directed acyclic graph” (DAG), so it does not contain any directed cycles.

## Activation functions

- ▶ Linear activation functions for the hidden units would lead to an overall linear model, because the composition of linear functions is a linear function!
- ▶ This is the reason why *some* non-linear function is required as activation function.
- ▶ A popular alternative to the logistic sigmoid is the tanh-function.

## Activation functions

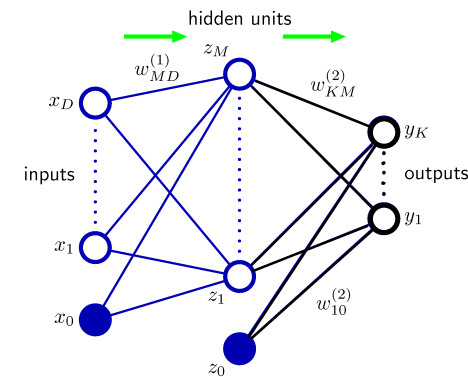
- ▶ The activation function in the output-layer determines the functionality of the network:
  - ▶ If we use a linear activation function, then the model will perform regression: It is simply a linear regression model applied to non-linearly preprocessed data.
  - ▶ If we use a single output unit with a logistic sigmoid activation function, we will perform (two-class) logistic regression.
  - ▶ Likewise, we can define a non-linear multiclass logistic regression model by using multiple outputs and combining them using

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{\exp(y_k(\mathbf{x}))}{\sum_j \exp(y_j(\mathbf{x}))}$$

## Learning arbitrary non-linear functions

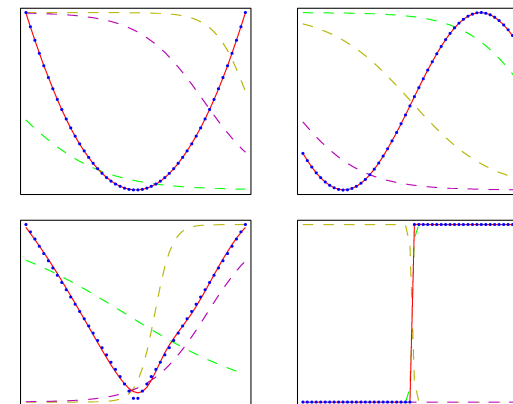
- ▶ It can be shown (Funahashi, 1989) that a network with a single hidden layer can model any non-linear function under fairly mild conditions to arbitrary accuracy.
- ▶ But the number of hidden units may need to be very large.
- ▶ Also, extremely large amounts of training data may be required, so the practical relevance of this result is limited.
- ▶ Even so, feed-forward neural networks can be very useful non-linear function approximators.

## Activation functions



- ▶ The last layer determines the functionality of the network.

## Learning arbitrary non-linear functions: Some 1-D examples



- ▶ Training data shown as blue dots.
- ▶ Hidden unit activations shown as dashed curves.

## Training

- ▶ To train a neural network for regression, given training data  $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n)\}$ , we take the usual approach:

- ▶ Minimize

$$\frac{1}{2} \sum_{n=1}^N \|\mathbf{y}(\mathbf{x}_n, \mathbf{w}) - \mathbf{t}_n\|^2$$

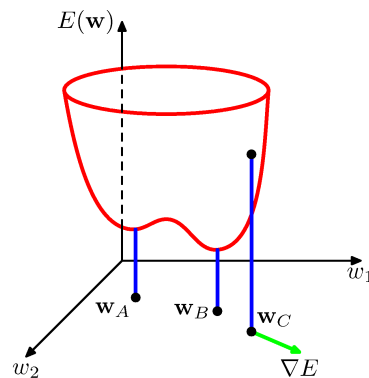
- ▶ Note that, as usual, we can interpret this as maximizing the likelihood of the data

$$p(\mathcal{D}) = \prod_n \mathcal{N}(\mathbf{t}_n | \mathbf{y}(\mathbf{x}_n, \mathbf{w}), a\mathbf{I})$$

with respect to the network parameters.

- ▶ The mean of these Gaussians is given by the output of the neural network.

## Training



- ▶ The objective function is non-convex, so there can be local minima.
- ▶ Practical approach: Use gradient descent to find parameters that are at least *good* (better than the initialization), even if they're not the global optimum.

## Training

- ▶ To train a neural network for (multi-class) classification, we also take the usual approach:

- ▶ Minimize

$$-\sum_{n=1}^N \sum_{k=1}^K t_{nk} \log p(\mathcal{C}_k | \mathbf{x})$$

where the network outputs are combined with the “softmax”-activation function to yield the probabilities:

$$p(\mathcal{C}_k | \mathbf{x}) = \frac{\exp(y_k(\mathbf{x}))}{\sum_j \exp(y_j(\mathbf{x}))}$$

## Backpropagation

- ▶ To perform gradient descent, we need to compute derivatives with respect to the parameters.
- ▶ There is a clever general-purpose procedure, that allows us to compute derivatives in networks of any topology and any depth.
- ▶ It is known as **error backpropagation** or simply **backprop**.

## Backpropagation

- First, let us recall that for any cost function which is the sum over point-wise costs:

$$E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w})$$

we can compute derivatives  $\frac{\partial E_n}{\partial \mathbf{w}}$  individually for each training case and add them up afterwards.

- We will now need to compute derivatives with respect to each weight  $w_{ji}$  connecting nodes  $i$  and  $j$  in the network.

## Backpropagation

- By the chain-rule of differentiation, we have

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial w_{ji}}$$

- The second factor is easy:

$$\frac{\partial a_j}{\partial w_{ji}} = z_i$$

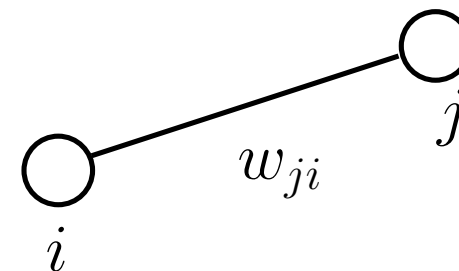
And all  $z_i$  are easy to compute: Just run the network!

- For the first factor, we define  $\delta_j := \frac{\partial E_n}{\partial a_j}$ , and rewrite the derivative as

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$$

- We now consider how to compute the  $\delta$ 's.

## Backpropagation



- To compute  $\frac{\partial E_n}{\partial w_{ji}}$  we use the following definitions:
- Let  $z_i$  be the activation of some unit  $i$  in the network.
- Let  $a_j = \sum_i w_{ji} z_i$  be the net input of an upstream unit  $j$  connected to unit  $i$ .
- Let  $z_j$  be unit  $j$ 's activation:  $z_j = h(a_j)$ .

## Backpropagation

- We apply the chain-rule once more to get

$$\delta_j = \frac{\partial E_n}{\partial a_j} = \sum_k \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j}$$

where the sum is over those units  $k$  which are connected to unit  $j$ .

- Intuitively, this reflects the fact that if we wiggle  $a_j$  this will affect the cost function through all the  $a_k$ .
- Using our definitions, this simplifies to:

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

- So we can use a recursion to compute all  $\delta$ 's, starting at the outputs!
- Note that  $h'(a_j) = h(a_j)(1 - h(a_j))$  if  $h$  is the logistic sigmoid, so we just need to use the  $z_i$  computed earlier.

## Backpropagation

- ▶ Computing the  $\delta$ 's for the outputs is easy, if that layer performs linear or logistic regression on the previous layer.
- ▶ For the squared error cost function (regression), we have

$$\frac{\partial}{\partial a_k} \frac{1}{2} \|\mathbf{y}(\mathbf{x}, \mathbf{w}) - \mathbf{t}\|^2 = y_k - t_k$$

since  $y_k = a_k$  in the case of regression.

- ▶ We get the same expression for the log-probability of the softmax activation function (exercise).

## Regularization

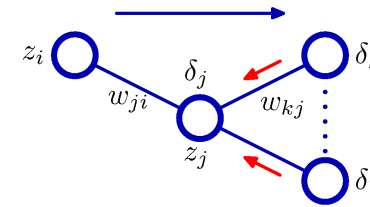
- ▶ Like any learning method, neural networks can overfit, so we have to regularize. It is common to add the usual penalty:

$$\frac{\lambda}{2} \|\mathbf{W}\|^2$$

where  $\mathbf{W}$  contains *all* network weights.

- ▶ The choice of the number of hidden units (and thus, the number of parameters) affects the model capacity, too:

## Backpropagation

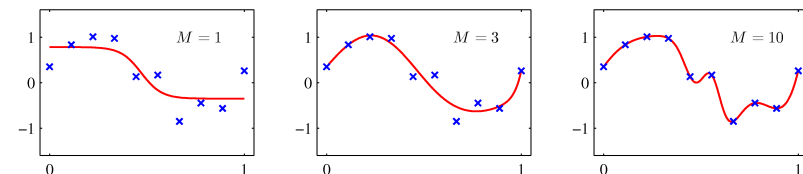


### Backpropagation summary (Bishop, page 244):

1. Apply an input  $\mathbf{x}_n$  to the network, forward propagate through the network to find activations of all hidden  $z_i$  and outputs.
2. Evaluate the  $\delta_k$  for all outputs.
3. Backpropagate the  $\delta$ 's to obtain  $\delta_j$  for each hidden unit.
4. Compute the derivatives for each  $w_{ji}$  in the network by multiplying the appropriate  $\delta_j$ 's and  $z_i$ 's.

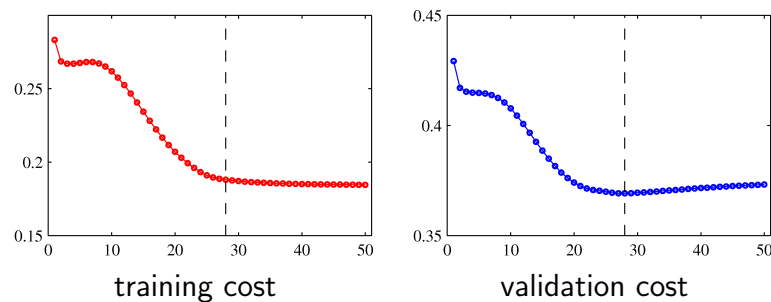
## Regularization: Example

- ▶ Example of a network with one hidden layer containing 1, 3 and 10 units, respectively, trained on 10 points from sinusoidal data:



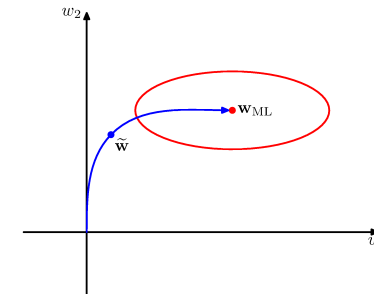


## Regularization: Early stopping



- ▶ An alternative to weight decay is **early stopping**:
- ▶ Typically, the cost on training data will decrease, while the cost on validation data will start increasing at some point.
- ▶ Early stopping amounts to picking the network that gives the best performance on the validation data.

## Regularization: Early stopping

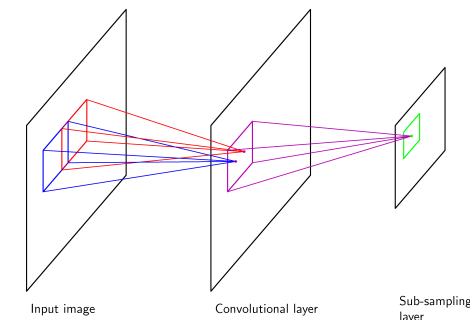


- ▶ Since parameters are usually initialized to small values for training, the effect of early stopping is similar to weight decay.
- ▶ Also, sigmoidal activations with *small* inputs behave much like linear activations, and a network composed of linear units has limited capacity (it is like a one-layer network).

## Weight sharing and convolutional networks

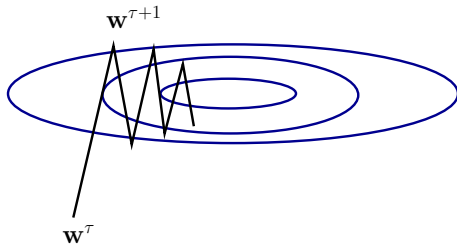
- ▶ A third common form of reducing model capacity in a neural network is **weight sharing**:
- ▶ Reduce the number of parameters in the model by forcing different parts of the network to use *the same* parameters!
- ▶ It is particular common in image processing: The content of images is typically *local*, in the sense that a particular feature may occur at any position within the image.
- ▶ So it would be wasteful to train the same weights multiple times to look at different positions of an image.

## Convolutional networks



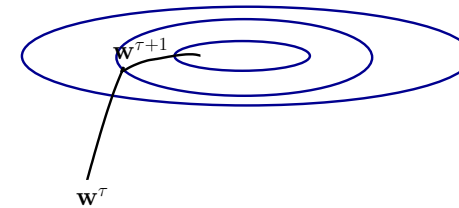
- ▶ Convolutional networks are common backprop networks that use local “receptive fields” with weight-sharing.
- ▶ They typically make use of additional, alternating sub-sampling layers to achieve better invariance with respect to small shifts.
- ▶ The most common application is image classification.

## Speeding up training with momentum



- ▶ When learning backprop networks with gradient descent, it frequently happens that convergence is slow because the presence of long, stretched-out “valleys” in the error surface leads to oscillations.

## Speeding up training with momentum

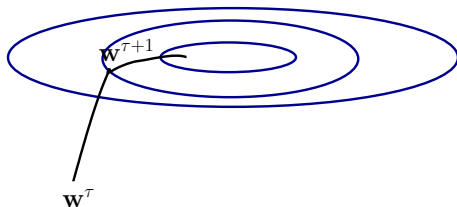


- ▶ A simple way to alleviate this problem is to add a “momentum term” to the update:

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \frac{\partial E_n}{\partial \mathbf{w}} + \mu \Delta \mathbf{w}(\tau)$$

where  $\Delta \mathbf{w}(\tau) = (\mathbf{w}^{(\tau)} - \mathbf{w}^{(\tau-1)})$  is the update from the last iteration, where  $\mu$  is set to a value between 0 and 1.

## Speeding up training with momentum



- ▶ The result is that the oscillations orthogonal to the valley are dampened.
- ▶ The accumulation of similar directions along the valley furthermore effectively leads to larger steps.
- ▶ The use of a momentum term is *highly* encouraged when performing gradient descent, because it typically yields a very significant speed-up in practice.

## Other types of neural network

- ▶ Backprop networks are only one of many types of neural network, but they are probably the most common ones in practical applications.
- ▶ Other examples include recurrent networks that contain feed-back connections (some of these can be trained with variations of backprop), Hopfield networks, Boltzmann machines, and many others.
- ▶ We will see some of these in future lectures.